

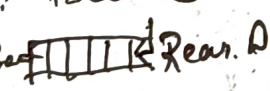
# DSA in Depth From Coding Section.

## Quality of video.

- ⇒ Intro to DSA
- ⇒ Abstract data type
- ⇒ linear search, binary search
- ⇒ Sorting Algo., bubble sort

## Intro to DSA:-

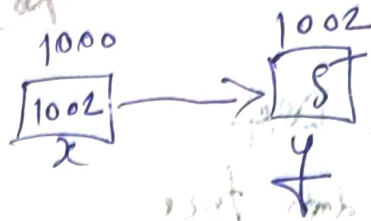
★ collection of data in some organized way  
Algo:- perform operation on data. logic:- way to do a task  
Algo ⇒ A step-by-step to solve a specific problem.

- ⇒ Pointers.
- ⇒ Structure.
- ⇒ Functions.
- ⇒ Dynamic Memory Allocation.
- ⇒ Asymptotic notation/complexity / Abstract Data type.
- ⇒ Searching & Sorting Techniques.  
linear / Binary / Bubble / Selection / Insertion / Quick / Merge / Bucket / etc.
- ⇒ Linked list :- Singly / Doubly / Circular.
- ⇒ Stacks :- FIFO
- ⇒ Queue :-  Rear. Dequeue.
- ⇒ Trees...
- ⇒ Graph

Pointer:-

```
int x;
int y;
x = &y;
*x = 5;
```

Static memory allocation



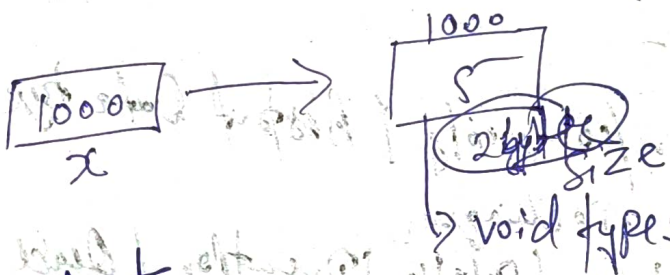
A variable that stores the memory address of another variable.

Dynamic Memory Allocation - memory allocate on run time

- malloc()
- calloc()
- realloc()
- free()

malloc(2);

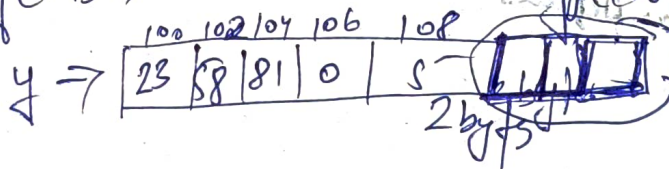
```
main() {
    int *x;
    x = malloc(2);
    *x = 5;
    printf("%d", *x);
}
```



malloc(size of int);  
(int\*) malloc(size of int);

Na of elements  
size of element  
y = calloc(5, size of int);

```
*y = 23;
*(y+1) = 58;
*(y+2) = 81;
*(y+4) = 5;
```



x = (int \*) malloc(size of int);

Size of 9 block  
after realloc

printf("%d", \*(y+3)); // 0 bcz if no data entered the default 0 assign

```
realloc(y, 8);
free(y);
```

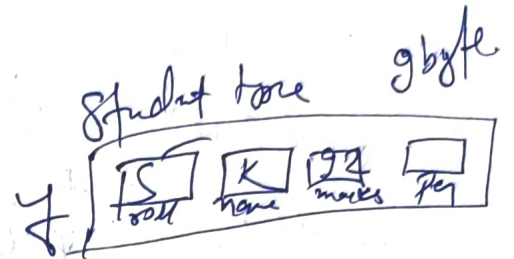
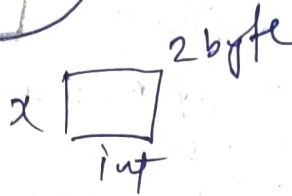


Structure! - Structure is a collection of different data types

```
main() {
    int roll;
    char name;
    int marks;
    float per;
}
```

```
struct Student {
    int roll;
    char name;
    int marks;
    float per;
};
```

```
main() {
    int x;
```



~~struct Student~~

struct Student y;

struct Student z;

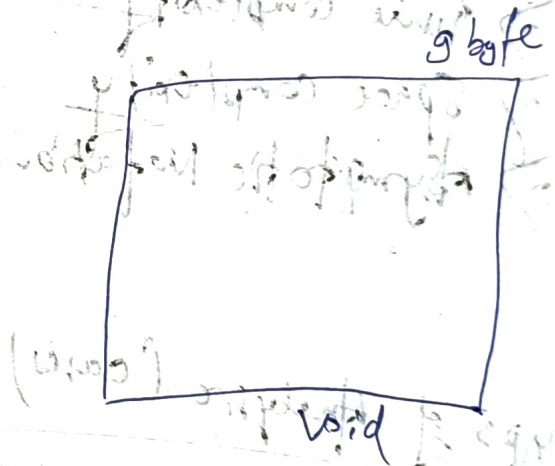
```
y.roll = 5;
y.name = 'K';
y.marks = 92;
y.per = 80.76;
```



```
struct Student *k;
k = (struct Student *) malloc (sizeof (struct Student));
```

```
k->roll = 1;
k->name = 'P';
k->marks = 52;
k->per = 81;
```

```
l = (struct Student *) malloc (sizeof (struct Student));
l->roll = 5;
printf ("%f", l->roll);
```



## Functions:-

add() {

≡

3

main() {

add(10);

add(1);

3

Structure:-

main()

{

Student \*start;

start = (Student) malloc (sizeof (Student));

start->roll = 5;

start->pu = 59.12;

start->k = (int) malloc (sizeof (int));

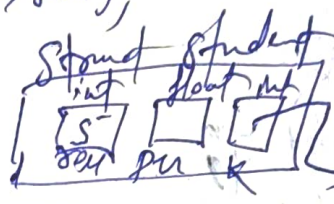
\*start->k = 15;

printf("id", start->roll);

getch();

3

start

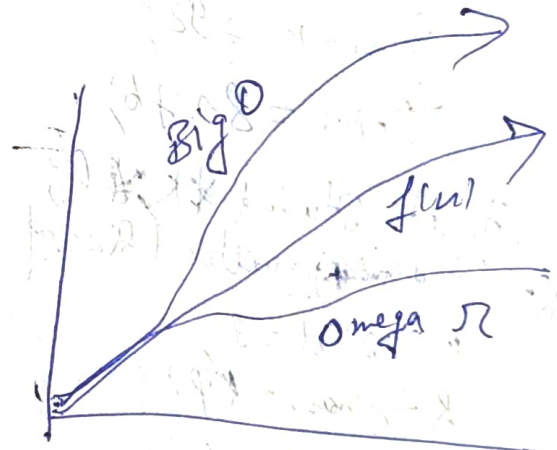


15

int

## Day:-3

- Algorithm
- Step by step solution of any problem to solve
- Time complexity
- Space complexity
- Asymptotic Notation



if both O notation

## Types of Analysis (cases)

1 2 3 4 5 6 7 8 9

1. Best case

2. Average case

3. Worst case

9 8 7 6 5 4 3 2 1

To analyze the given algorithm we need to know on what inputs the algorithm is taking less time and what input taking a huge time

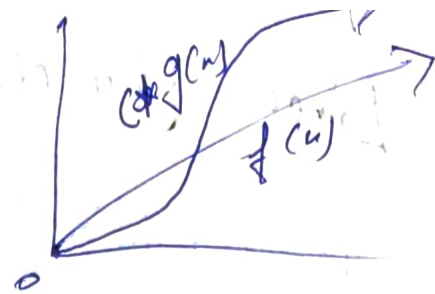


$$f(n) = n^2 + 3n + 1$$

$$2g(n) = 2n^2$$



| $n =$  | 1 | 2  | 3  | 4  |
|--------|---|----|----|----|
| $f(n)$ | 5 | 11 | 19 | 29 |
| $g(n)$ | 2 | 8  | 18 | 32 |

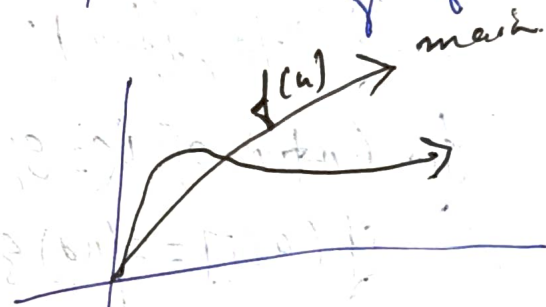


B-O notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$

Such that  $0 \leq f(n) \leq cg(n)$  for all  $n \geq n_0$

— The statement  $f(n) = O(g(n))$  states only that  $g(n)$  is an upper bound on the value of  $f(n)$  for all  $n, n \geq n_0$ .



Abstract Data Type

A data with related data operation on it is ADT.

⇒ A ADT is a mathematical model that defines a data type by what operations can be performed on it, without specifying how those operations are implemented.

# Linear Search (Sequential Search)

```
#include <stdio.h>
```

```
int main() {
```

```
    int find, a[20];
```

```
    printf("Enter 10 digits ");
```

```
    for (int i=0; i<=9; i++) {
```

```
        scanf("%d", &a[i]);
```

```
    }
```

```
    printf("Enter a number that you want to search: ");
```

```
    scanf("%d", &find);
```

```
    for (int i=0; i<=9; i++) {
```

```
        if (a[i] == find) {
```

```
            printf("Number found at index: %d", i);
```

```
        }
```

```
    }
```

## Binary Search

```
#include <stdio.h>
```

```
void BinarySearch(int n, int a[20], int search) {
```

```
    int k=0;
```

```
    int l=0;
```

```
    int r=n-1;
```

```
    while (l<=r) {
```

```
        int m=(l+r)/2;
```

```
        if (a[m] == search) {
```

```
            printf("Number found at position: %d", m+1);
```

```
            k++;
```

```
            break;
```

else if ( $a[m] > \text{search}$ ) {

$r = m - 1;$

} else if ( $a[m] < \text{search}$ ) {

$l = m + 1;$

}

}

if ( $R == 0$ ) {

printf("Element not found\n");

}

}

void main() {

int n;

int search;

int a[20];

printf("Enter the total number of element: ");

scanf("%d", &n);

printf("Enter %d digits: ", n);

for (int i = 0; i < n; i++) {

scanf("%d", &a[i]);

}

printf("Enter a num that you want to search: ");

scanf("%d", &search);

BinarySearch(n, a, search);

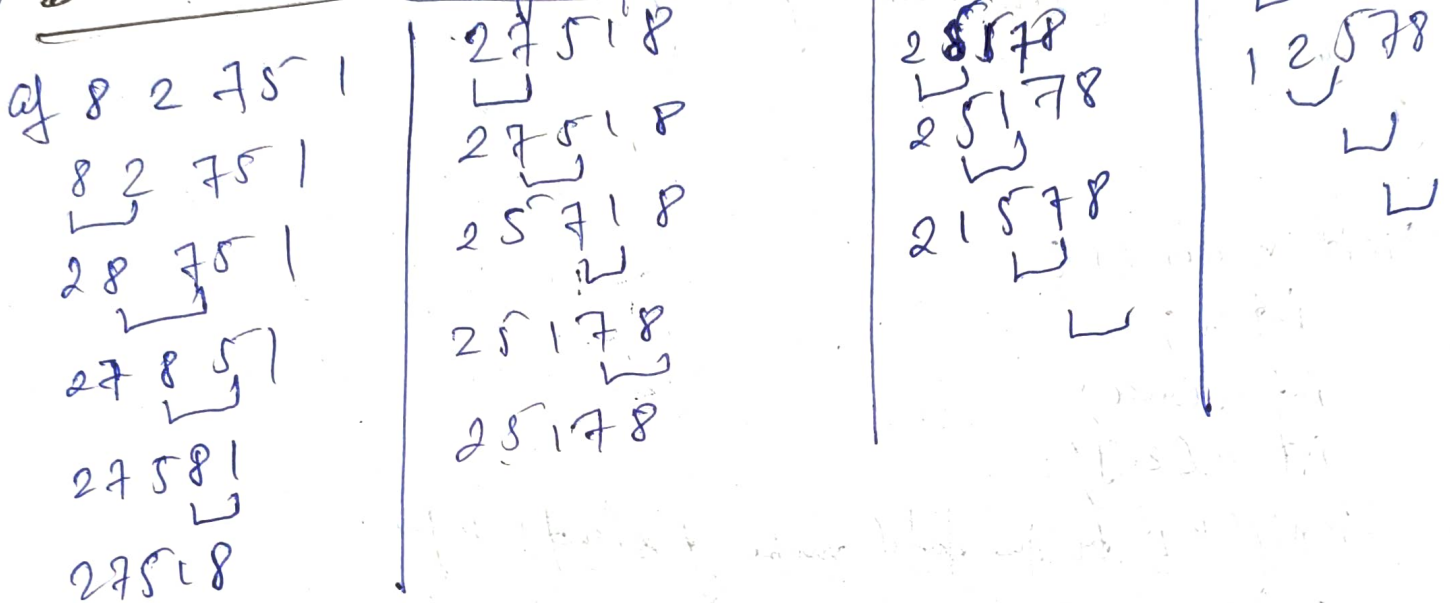
}



# Sorting

⇒ The arrangement of data in some logical order.  
 If the data is number then we arrange it in ascending or descending order of the data.  
 If characters then arrange it in dictionary order.  
 This is all sorting.

⇒ Bubble Sort  $\rightarrow O(n^2)$  very inefficient



#include <stdio.h>

```
int main() {
    int arr[5] = {40, 1, 2, 5, 6, 3, 77, 6, 8};
    for (int j=0; j<8; j++) {
        for (int i=0; i<7; i++) {
            if (arr[i] > arr[i+1]) {
                int temp = arr[i];
                arr[i] = arr[i+1];
                arr[i+1] = temp;
            }
        }
    }
}
```

```
for (int i=0; i<8; i++) {
    printf("%d, ", arr[i]);
}
```



# Selection Sort

Step 1:

9 1 2 3 8  
min = 9

9 1 2 3 8

compare

min = 1 9 1 2 3 8

min = 1 9 1 2 3 8

9 1 2 3 8

Step 2

1 9 2 3 8

1 9 2 3 8

1 9 2 3 8

min = 2

1 9 2 3 8

1 2 9 3 8

1 2 9 3 8

1 2 3 9 8

1 2 3 9 8

1 2 3 8 9

#include <stdio.h>

int main() {

int a[10] = {10, 3, 23, 1, 20, 5, 3, 7, 1, 10};

for (int i = 0; i < 5; i++) {

int min = a[i];

int pivot = i;

for (int j = i + 1; j < 6; j++) {

if (min > a[j]) {

min = a[j];

pivot = j;

3 3

int temp = a[i];

a[i] = a[pivot];

a[pivot] = temp;

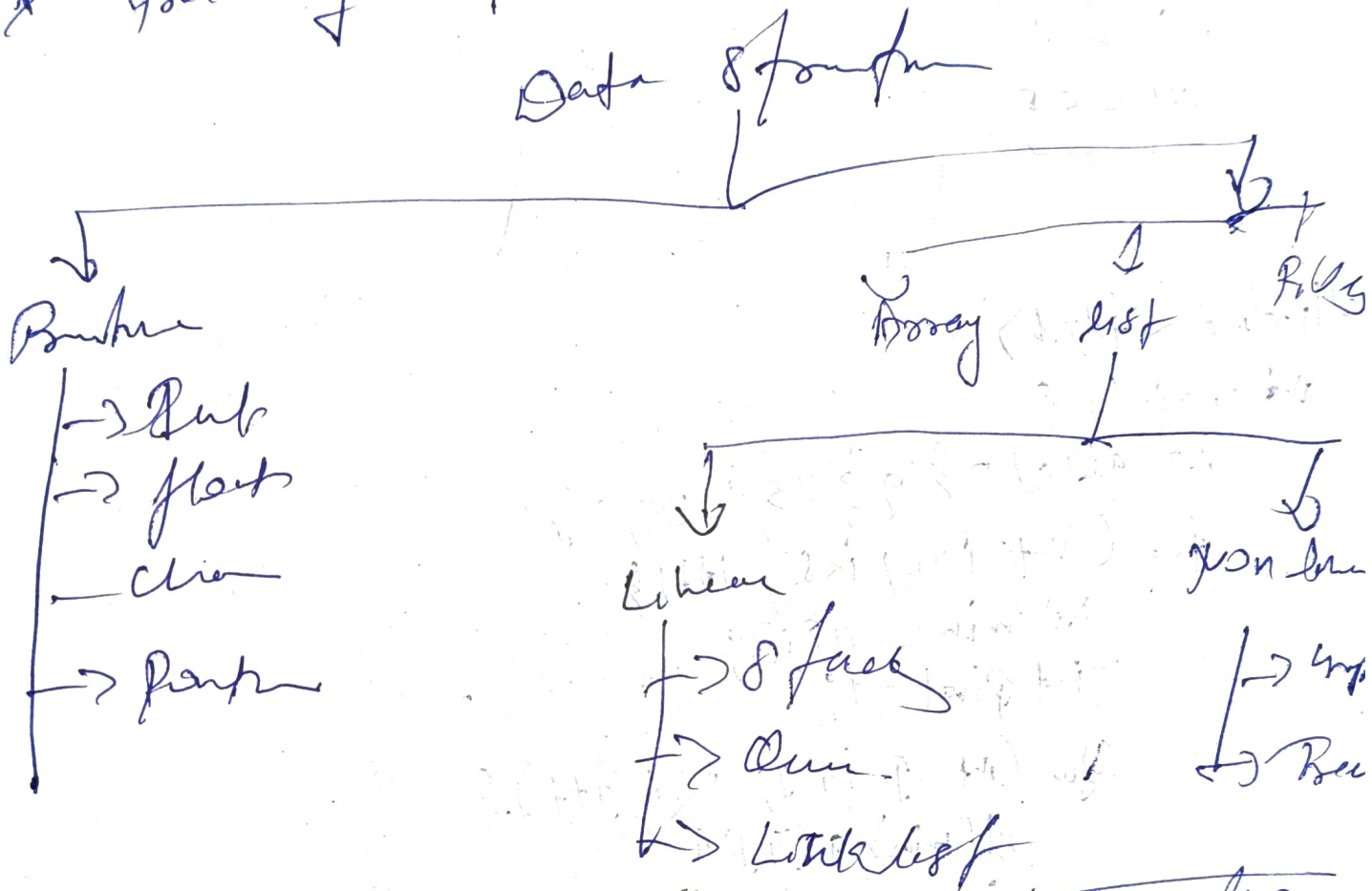
// Print array loop & Print Transverse

9

# Data Structure

⇒ Data Structure mainly specifies the following four things.

- ★ Organization of data How?
- ★ Accessing methods.
- ★ Degree of association
- ★ Processing methods.



Array:

Efficient storage  
Random access  
Easy to sort  
Flexibility  
Easy to use

at retrieval  
fast access  
and search

disadv

- ① Fixed size.
- ② Heterogeneous data
- ③ Poor performance for some operation
- ④ No built-in support for insertion or deletion

## Applications of Array

- ① memory management
- ② Data Representation
- ③ Database management
- ④ implementing Data Structure
- ⑤ Caching & Buffering

No. of elements  $= (ub - lb) + 1$

upper bound. lower bound

Size = no. of elements  $\times$  size of each element.

## Sparse Matrix

Max element  $> 0$   
Zero

Min 0  $\Rightarrow$  dense  
max 0  $\Rightarrow$  sparse.

## Linked list

Struct node {

int data;

Struct node \* next;

};

Adv: Dynamic size and efficient memory usage

$\rightarrow$  Fast insertion and deletion

$\rightarrow$  Versatility.

Disadv:

Slow access time

memory overhead

pointer manipulation



| <u>Array</u>     |                                                  |
|------------------|--------------------------------------------------|
| Memory           | Contiguous memory allocation                     |
| Size             | Fixed.                                           |
| Access time      | $O(1)$ direct access due to indexing             |
| Insert/Delete    | $O(n)$ in worst case<br>shifting may be required |
| Memory efficient | More memory efficient for known size of data     |

Linked list

Non-contiguous memory location

Dynamic size

$O(n)$  for accessing an element as it requires traversal

$O(1)$  if the pointer to the node is known

Extra memory for pointers

## Stacks:-

Applications:-

Expression parsing.  
 Back tracking.  
 function calls,  
 undo feature  
 Syntax checking.

$push(S, N, Top, x) \{$

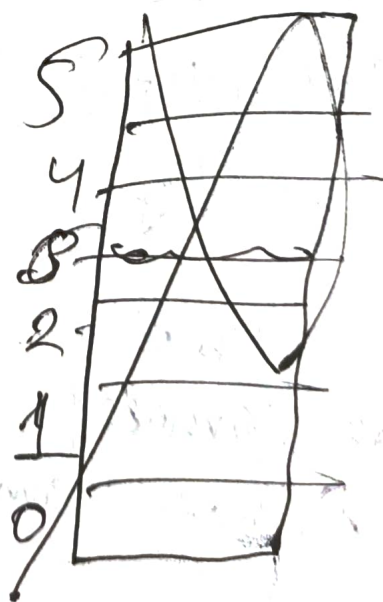
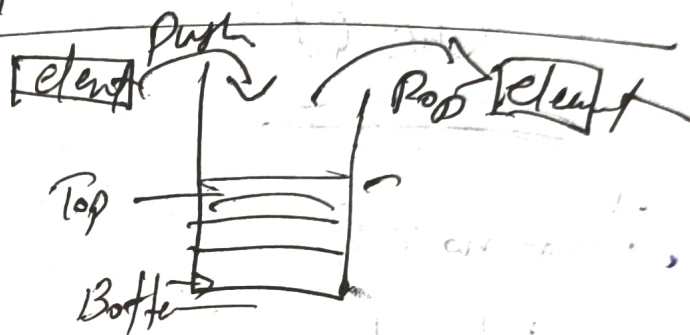
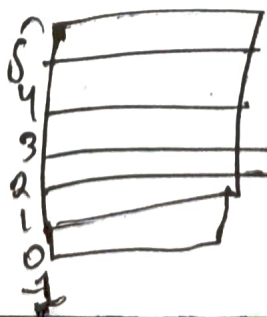
$\quad if (Top \geq N-1) \{$

$\quad \quad \text{Print (Stack is overflow)}$

$\quad \quad \rightarrow Top = Top + 1$

$\quad \quad S[Top] = x$

$\quad \quad \rightarrow \text{exit}$



POP (S, V, TOP) {

if (TOP == -1)

Pop underflow.

V = S[TOP]

TOP = TOP - 1

return V; exit

}

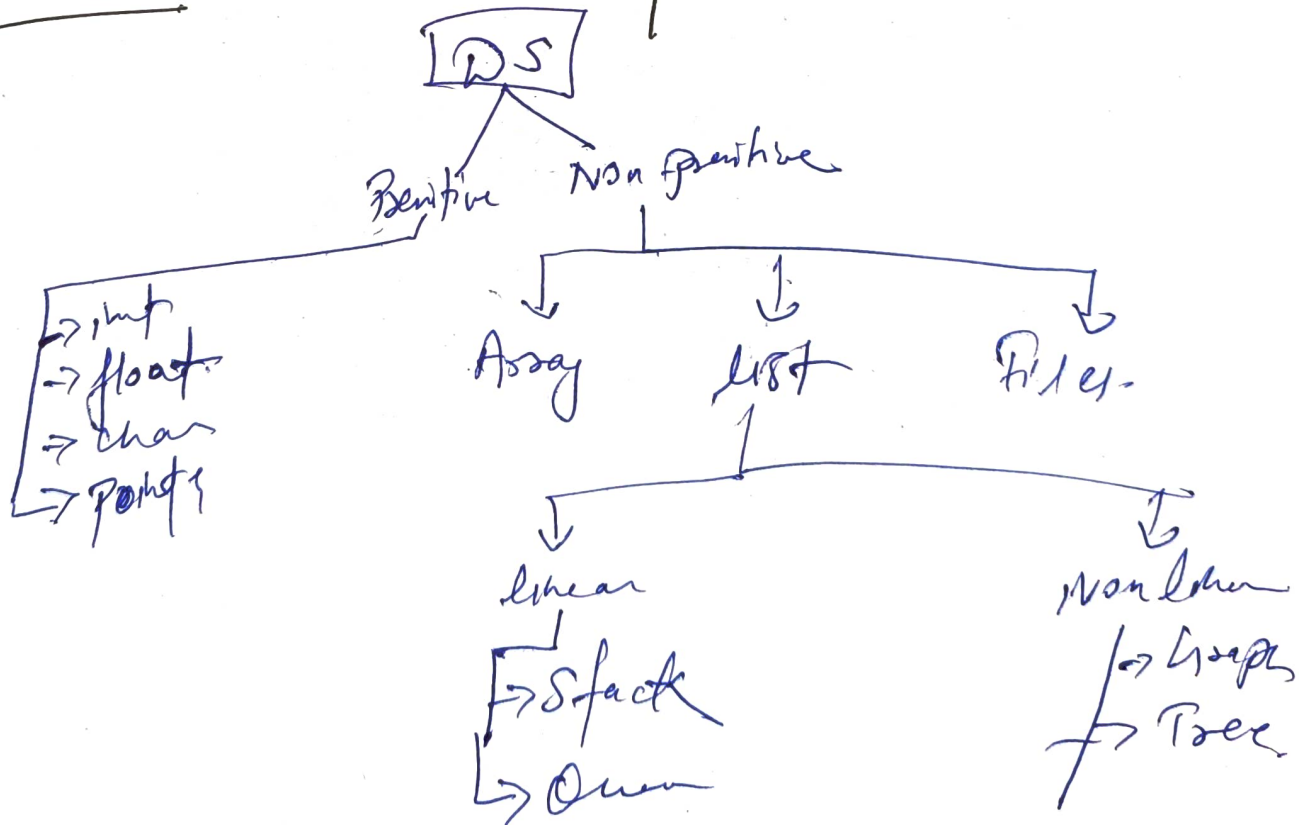
Prefix:- A + B.

Prefix:- + AB.

Postfix:- AB +

is Cube Tech

Data Structure Algo.



1871  
1872  
1873  
1874  
1875

1876  
1877  
1878  
1879  
1880

1881

1882  
1883  
1884  
1885  
1886  
1887  
1888  
1889  
1890

1891  
1892  
1893  
1894  
1895



# DS (Data Tech) Data Structures Algo.

Data:- Raw facts, Unorganized facts. that need to process data  
Info:- when data is processed organized, structured presentation

Algo:- Step by step instruction to solve a specific problem

Program:- An implementation of Algo. in some programming language.

Data Struct:- Organizing data needs to solve problem

Pseudocode:- In algo we can use some loop condition if else. write using programming concepts.

Type of Data structure:- DS is a way to collecting or organizing data such way to perform an operation efficiently way

Types of DS:- Primitive and non-primitive.  
Primitive: int, float, char, pointers  
Non-primitive: Array, List, Queue, Stack, Tree, Graph, Hash, Binary, etc.

Primitive:- Predefined. operate on the machine instructions.  
non-primitive:- using primitive DS create a new DS by us column

Linear:- arranged in sequence. left or right, having in single line.  
Non-linear:- attached with each other. not in sequence. implement difficult.

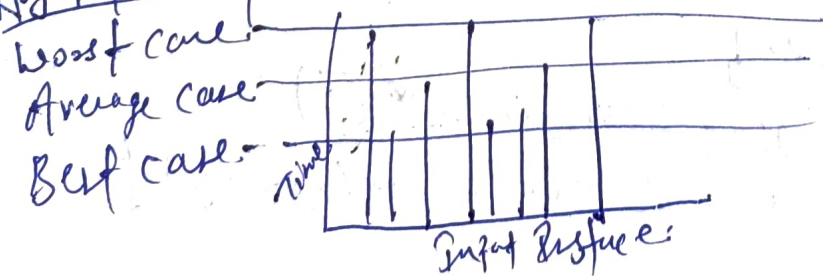
Static:- fixed size. Array structure fixed.

Dynamic:- expand or shrink upon need. program execution.

Homogeneous:- similar type of data if int then only int  
Non-Homogeneous:- different type of data. we int float char of same type.

Efficiency:- Running time, Space used.

Asymptotic Notation



Signature

# Asymptotic Analysis

$$f(n) = 8n^2 + 6n + 10$$

$$\boxed{f(n) = n^2}$$

Two Basic rules:-

1. Drop all lower order terms.
2. Drop all constants.

Asymptotic Notation mathematical representation

$f(n) \leq (g(n))$  Big - Oh ( $O$ )      upper bound worst case max time  
 $f(n) \geq (g(n))$  Big - Omega ( $\Omega$ )      Best case. lower bound min time  
 $ag(n) \leq f(n) \leq bg(n)$  Big - Theta ( $\Theta$ )      Average case. tight bound. Average time

## Common Asymptotic Notations:

$O(1)$  - constant

$O(\log n)$  - logarithmic

$O(n)$  - linear

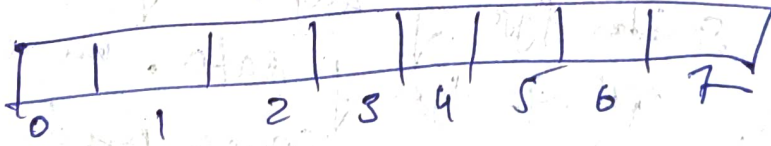
$O(n^2)$  - quadratic

$O(n^c)$  - Polynomial or algebraic

$O(c^n)$ ; where  $c > 1 \rightarrow$  exponential.

Array: It is a collection of similar type of data. Homogeneous linear

Lower bound 0  
 Upper bound 7  
 length: 8  
 int arr[10];



int arr[7] = {0, 20, 3}

int arr[6] = {0, 20, 20}

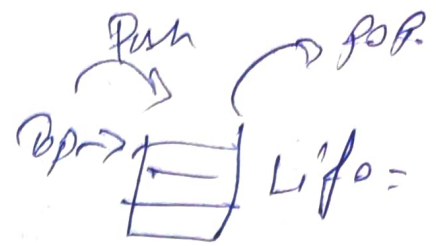
$$LOC(A[i]) = \text{Base address} + w * (i)$$

$$LOC(A[i:j]) = \text{Base Address} + w [M(i) + (j)]$$



|            | Array  | Search | Insert | Delete |
|------------|--------|--------|--------|--------|
| Array      | $O(1)$ | $O(n)$ | $O(n)$ | $O(n)$ |
| LinkedList | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ |

Stack: linear data structure.  
delete from top. One end.



Push

Pop

Top

is Empty.



underflow empty

using Array (Static)  
using LinkedList (Dynamic)

Stack using Array

PUSH Stack (Stack, Top, max, Item.)

if  $Top == Max$  {  
Print:- Stack is overflow return

$Top = Top + 1;$

$Stack[Top] = Item;$

exit.

POP Stack (Stack, Top, Item)

if  $Top == -1$ ;

Print:- Stack is empty return

$Item = Stack[Top];$

$Top = Top - 1;$

exit

Applications of Stack

\* Balancing Symbol

\* Redo undo.

\* find backing browser

\* Tower of Hanoi, percolation  
DFS.

\* Prefix, infix, postfix  
expression

\* reverse a string

\* function calling.



$$(A+B) + (C+D) / h$$

Proof:

|   |     |           |
|---|-----|-----------|
| ( | (   |           |
| A |     | A         |
| + | (+  |           |
| B | (+) | AB        |
| ) |     | AB+       |
| + | +(  |           |
| C |     | AB+C      |
| + | +(  |           |
| D | +   | AB+CD     |
| ) |     | AB+CD-    |
| / | +   |           |
| h | +/  | AB+CD-h/+ |

$$(A+B) + (C-D) / h$$

$$\{+AB\} + \{-CD\} / h$$

$$+AB + 1-CDh$$

$$++AB / -CDh$$

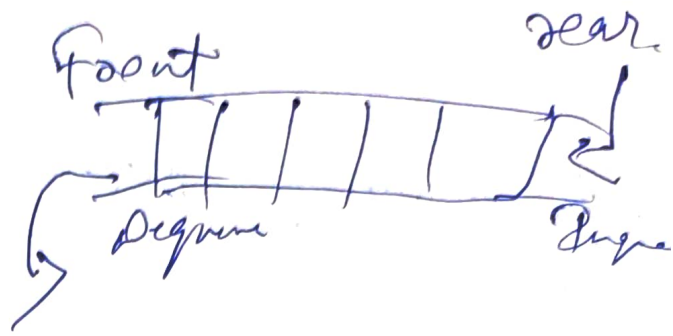


## Queue

Linear Data Structure  
Restricted DS.

Based on FIFO Concept

Rear and Front two pointer  
for insertion and deletion



Insert (Q, front, Rear, Max, Item).

① if  $(Rear == Max - 1)$  {  
    Queue is over flow.  
    return.

② if  $(front == -1 \text{ and } rear == -1)$  {  
    front = 0  
    rear = 0.

else  $= Rear + 1$

③  $Q[Rear] = Item$ .

④ Print Successfully  
    end

## Deletion

① if  $(front == -1)$  or  $(front == rear)$  {  
    Queue is underflow.  
    return

②  $Item = Q[front]$

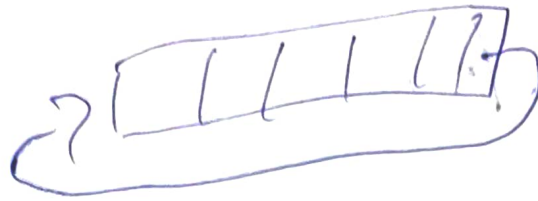
③ if  $(front == rear)$

    front = -1  
    rear = -1

else  
    front = front + 1

④ Deleted Successfully

Circular Queue:



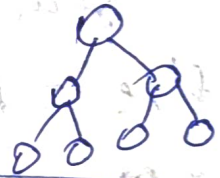
Binary tree - max child 2.

full binary / strictly per.  
child:- 2 ya 0

Complete binary tree

insertion left to right.

Perfect binary tree - last level second level

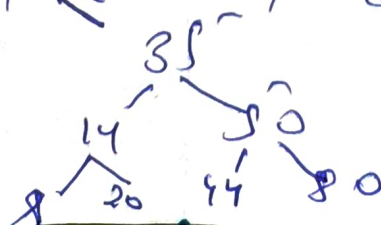


Preorder:- Root, Left subtree, Right subtree

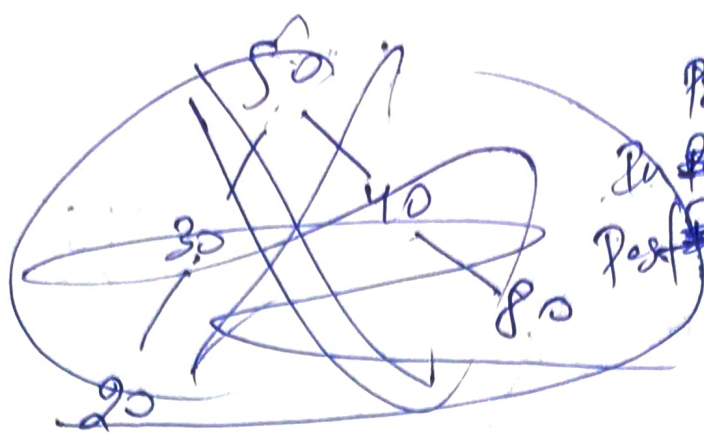
Inorder:- left, root, right

Postorder:- left right root

BST:-  $\text{left} \leq \text{Data} \leq \text{right}$

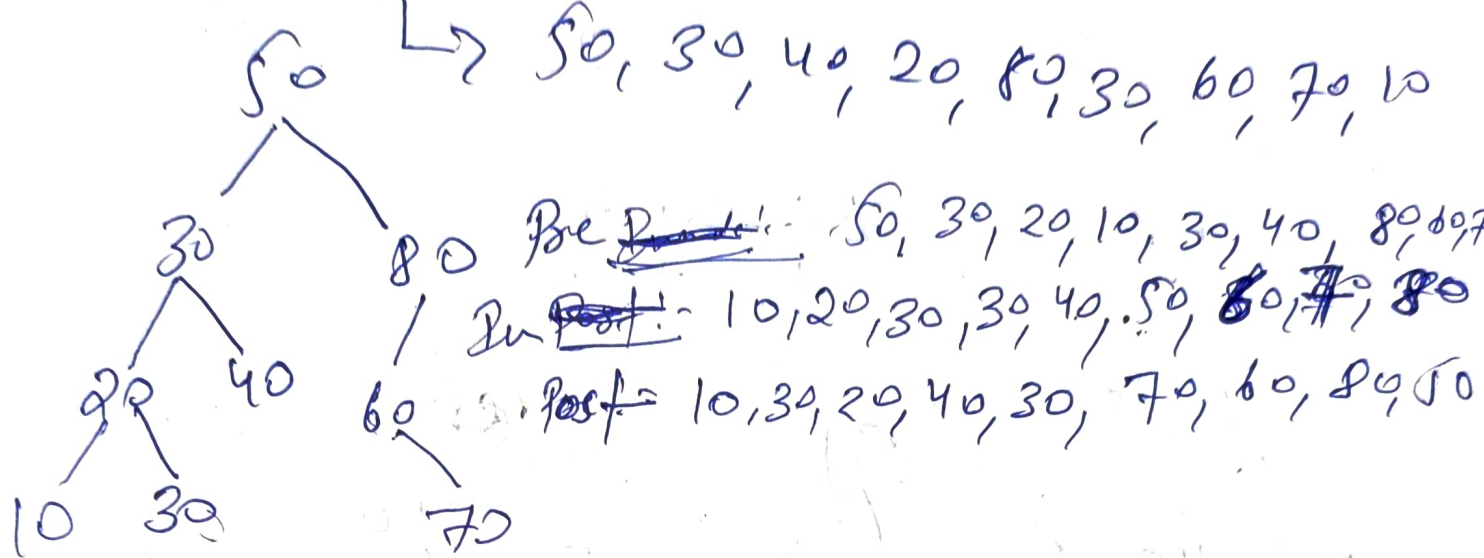






Pre ~~Root~~ :- Root, R  
 In ~~Root~~ :- L, Root, R  
 Post ~~Root~~ :- L, R, Root

BS T !

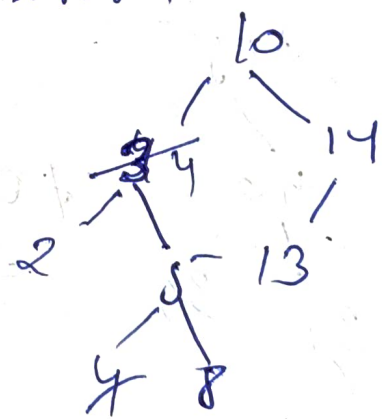


Deletion - ① Leaf node - Direct deletion

② One child. delete and child send to ~~Root~~ parent

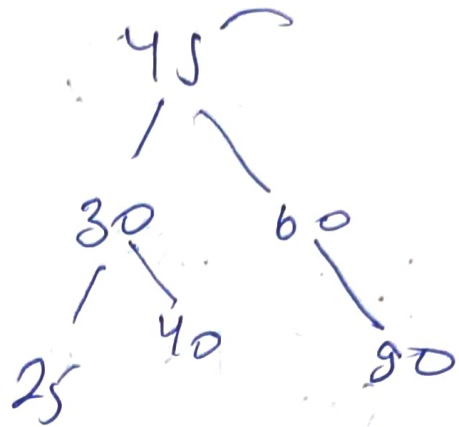
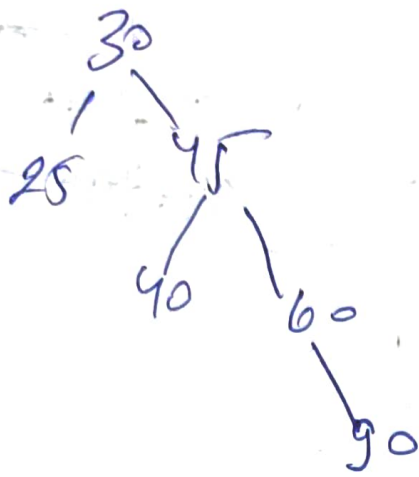
③ Two child. Inorder. Just successor.

Example: 2, 3, 4, 5, 8, 10, 14, 13.

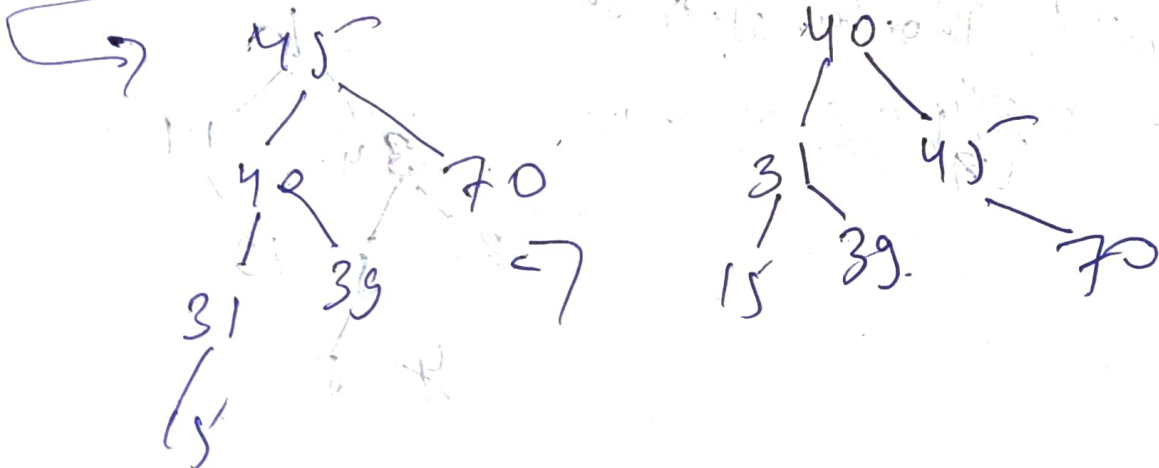
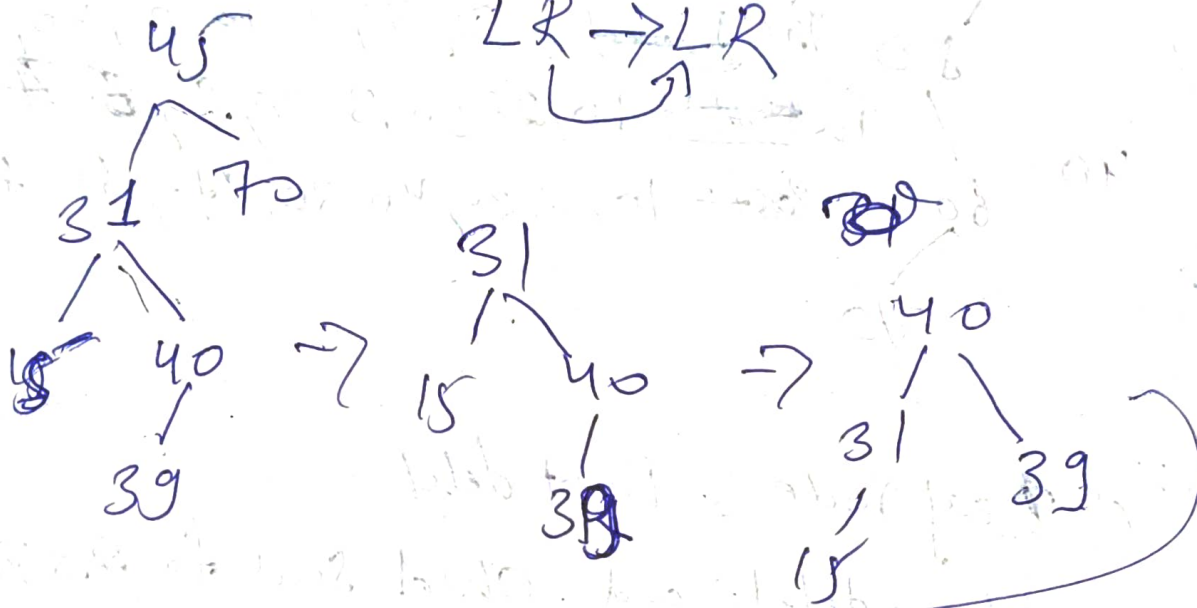


Pre: Root, left, right  
 In: left Root, right  
 Post: left right, root

RR  $\rightarrow$  L

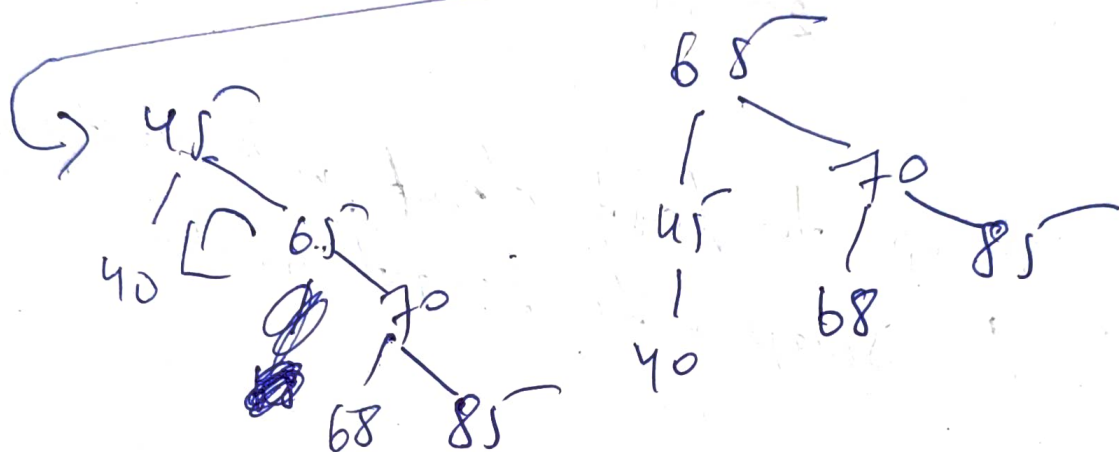
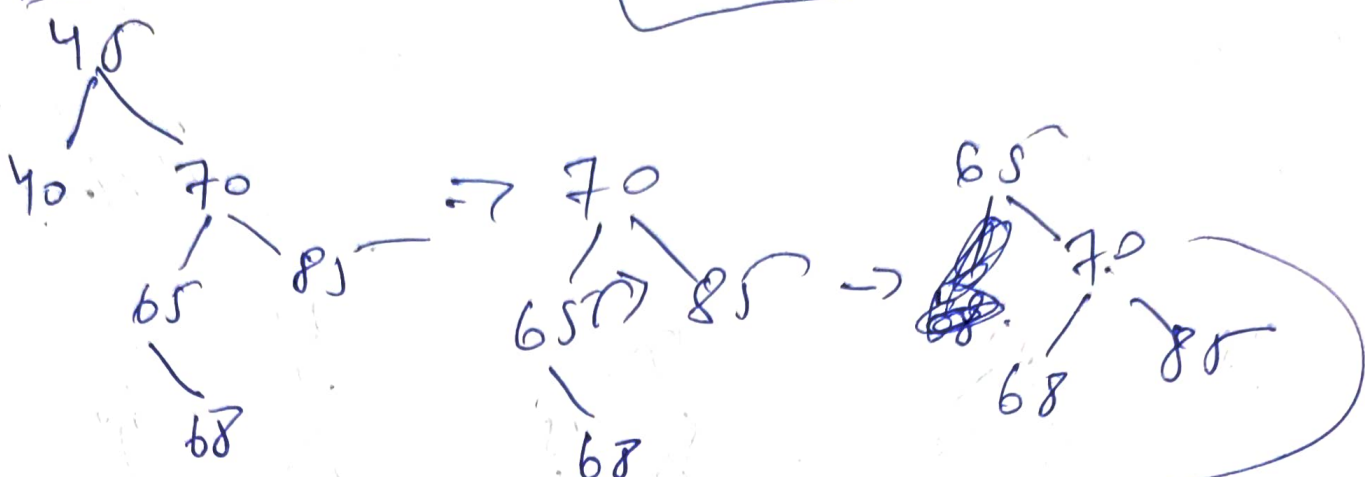


LR  $\rightarrow$  LR

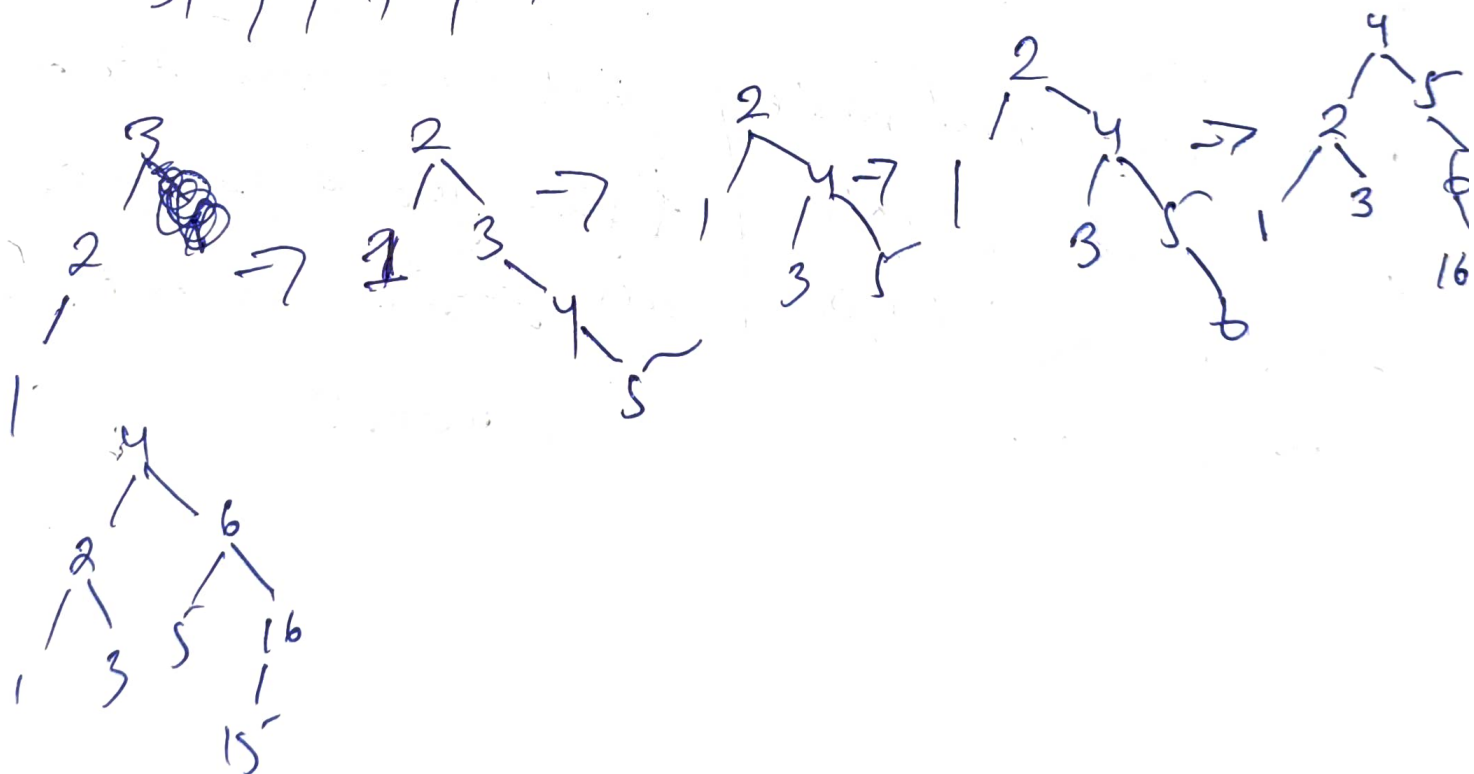


RL  $\rightarrow$  R

1-3-2



3, 2, 1, 4, 5, 6, 16, 11



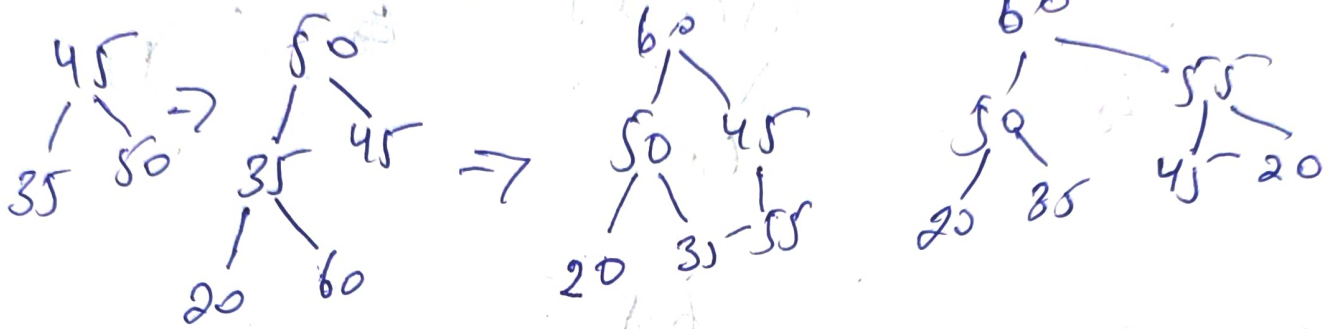


Heap: Heap is a complete binary tree.  
max - or min

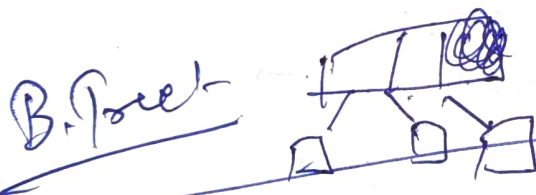
max Heap

45, 35, 50, 20, 60, 55, 20

left to right  
fill element.



Delete Heap: Swap to last element, with delete data.  
 then delete it, then swap with largest number.



B-Tree

Tree  
 unique node root node  
 there is not any cycle  
 all nodes are graph  
 hierarchical

Graph

all nodes are root  
 cycle formed or not  
 all graphs are not tree  
 network.

## ⇒ Data Structure

It is a process of storing the data in computer memory. In such a way so that we can access it easily and efficiently.

- ① Store
- ② Organize
- ③ Access.
- ④ Manipulate

### ⇒ Linear DS

- ⇒ Adjacently Attached
- ⇒ Single level.
- ⇒ Easy
- ⇒ Single Run
- ⇒ Memory is not utilized in efficient way.
- ⇒ Array, Stack, LL.

### Non Linear DS.

- ⇒ Hierarchically Attached
- ⇒ multiple level
- ⇒ Difficult
- ⇒ multiple Runs.
- ⇒ In efficient way
- ⇒ Tree Graph.

## ③ Array:-

store  
Same type  
contiguous memory locations.  
Index Accessing

